Hashování

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

26.03.2026

CC BY-NC-SA 4.0

Hashovací tabulka

Motivace

Chceme datovou strukturu, která umí rychle **vyhledávat**, **vkládat** i **mazat**

Co už známe?

Operace	**Pole (neseřazené)**	**Pole (seřazené)**	**Spojový seznam**
Vyhledání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(n)$
Vložení	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$

Dá se mít **všechno $\mathcal{O}(1)$ / $\mathcal{O}^*(1)$**?

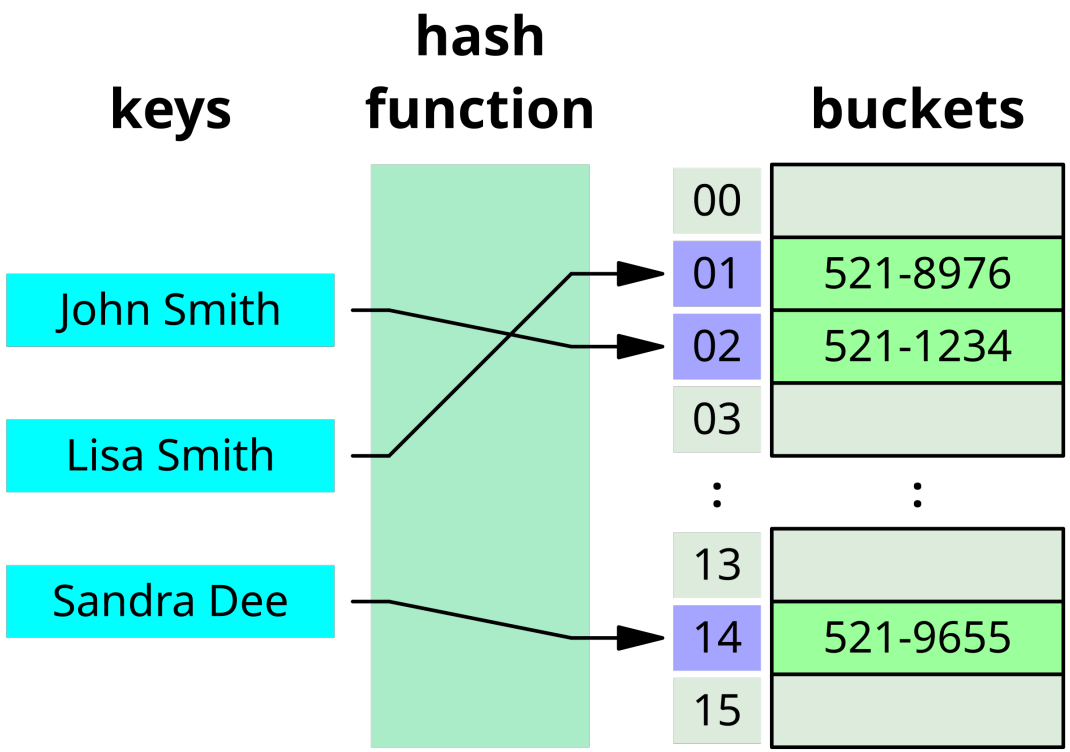
Princip hashovací tabulky

Hashovací tabulka ukládá **hodnoty** do paměti na základě **klíče**

Princip: klíč → **hashovací funkce** → index v poli

Přímý přístup k prvku – bez procházení, bez porovnávání

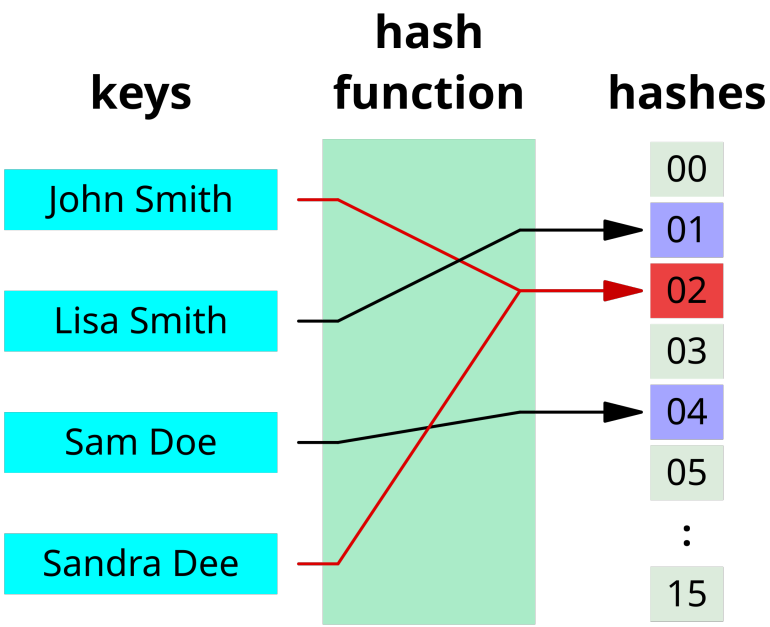
Princip hashovací tabulky



Zdroj: Wikipedia – *Hash table*

Hashovací funkce

Hashovací funkce převádí klíč na číselný index



Zdroj: Wikipedia – *Hash function*

Vlastnosti hashovací funkce

Jaké vlastnosti by měla mít hashovací funkce pro tabulku?

- **Deterministická** – stejný vstup = stejný výstup
- **Rychlá** – efektivní výpočet
- **Uniformní distribuce** – rovnoměrné rozložení hodnot
- **Minimalizuje kolize** – různé vstupy $\rightarrow$ různé indexy

Příklad jednoduché hashovací funkce

```
unsigned int simple_hash(const char *key, int table_size) {
    unsigned int hash = 0;
    for (int i = 0; key[i] != '\0'; i++) {
        hash += key[i];
    }
    return hash % table_size;
}
```

`"abc"` má ASCII součet = 97 + 98 + 99 = 294

Pro tabulku velikosti 10: `294 % 10 = 4`

Příklad jednoduché hashovací funkce

Jaký je problém této hashovací funkce?

`"abc"` a `"cab"` mají **stejný** ASCII součet = 294

V praxi se používají složitější funkce, které zohledňují **pozici** znaku

Populární hashovací funkce pro tabulky: **djb2**, **FNV-1a**, **MurmurHash**, …

Kolize

Co je kolize?

Pro daný klíč k vypočteme index $i = h(k) \bmod m$

Ale na indexu i již existuje **jiný** prvek

Existuje-li x a y takové, že $x \neq y$ a $h(x) = h(y)$, jedná se o **kolizi**

Jsou kolize nevyhnutelné? **Ano** – mapujeme potenciálně nekonečnou množinu klíčů na konečný počet indexů (princip holubníku)

Řešení kolizí

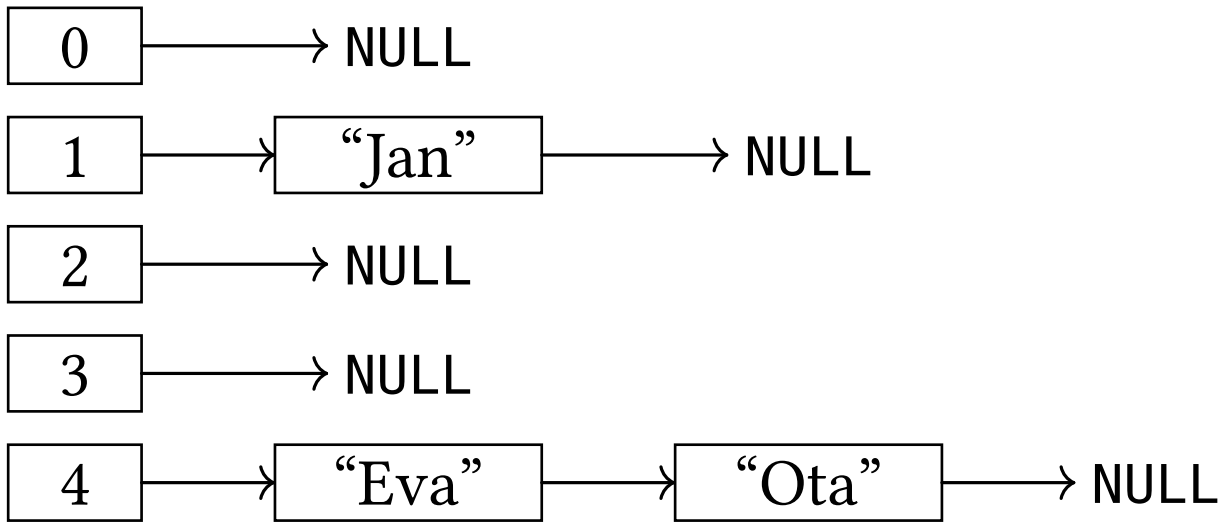
Jak kolizi vyřešit?

- **Řetězení** (chaining)
- **Otevřené adresování** (open addressing)

Řetězení (chaining)

Princip řetězení

Na každém indexu pole uchováváme **spojový seznam** prvků



Výhody a nevýhody

Výhody řetězení:

- Jednoduchá implementace
- Tabulka se nikdy “nezaplní”

Nevýhody:

- Při vyhledávání je nutné procházet seznam
- Horší paměťová lokalita (pointery na spojový seznam)

Otevřené adresování

Princip otevřeného adresování

Pokud je index i již obsazen, hledáme **další volný index** přímo v tabulce

Provádíme takzvaný **probing** (sondování)

Jakým způsobem hledáme další volný index?

- **Lineární probing**
- **Kvadratický probing**
- **Dvojité hashování**

Lineární probing

Pokud je index i již obsazen, zkoušíme: $i + 1$, $i + 2$, $i + 3$, ...

```
// Table size 10, hash function: key % 10
insert(23);  // h = 3, stored at index 3
insert(13);  // h = 3, collision! -> index 4
insert(33);  // h = 3, collision! -> index 5
insert(43);  // h = 3, collision! -> index 6
```

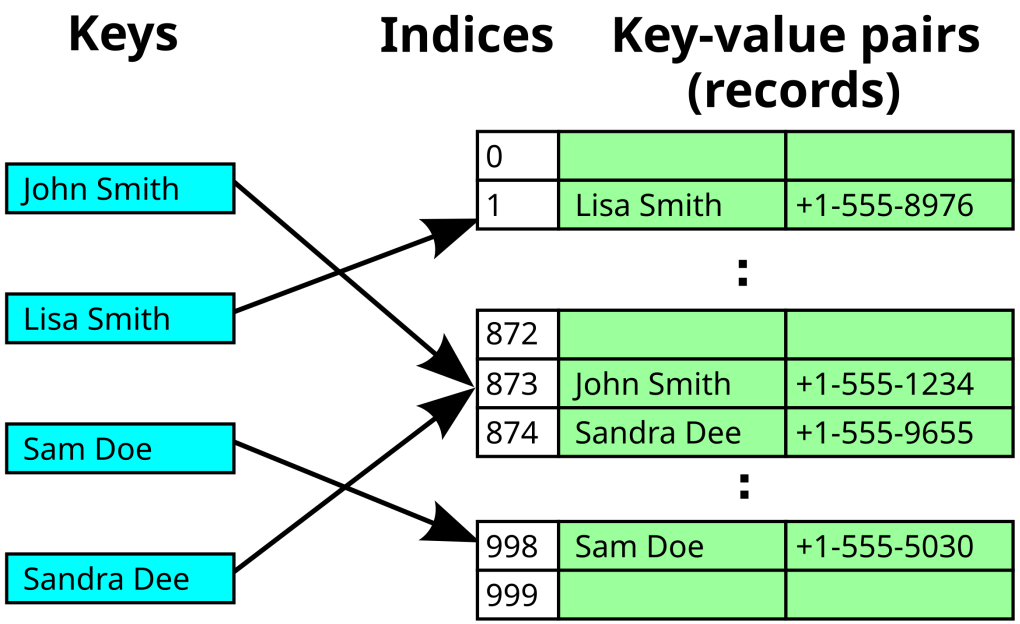
Vzniká **cluster** na indexech 3–6

Lineární probing

Jaký je problém lineárního probingu?

- **Clusterování** = shlukování prvků – čím větší cluster, tím větší pravděpodobnost dalšího růstu
- **Přetečení** = překročení velikosti pole

Lineární probing



Zdroj: Wikipedia – *Linear probing*

Kvadratický probing

Pokud je index i již obsazen, zkoušíme: $i + 1^2$, $i + 2^2$, $i + 3^2$, ...

```
// hash(key) = 3, table size 10
(3 + 0*0) % 10  // = 3
(3 + 1*1) % 10  // = 4
(3 + 2*2) % 10  // = 7
(3 + 3*3) % 10  // = 12 % 10 = 2
(3 + 4*4) % 10  // = 19 % 10 = 9
```

Výhoda: menší clusterování než u lineárního probingu

Dvojité hashování

Použijeme **dvě různé** hashovací funkce

- $h_1(k)$ určí základní index
- $h_2(k)$ určí velikost kroku při kolizi

Posloupnost: $h_1(k)$, $h_1(k) + h_2(k)$, $h_1(k) + 2 \cdot h_2(k)$, ...

Důležité: $h_2(k)$ nesmí být 0 a mělo by být nesoudělné s velikostí tabulky

Dvojité hashování

Co znamená **nesoudělné**?

Dvě čísla jsou nesoudělná, pokud jejich **největší společný dělitel** je 1

Příklad: tabulka velikosti $m = 8$, krok $h_2(k) = 4$

Pozice: $0, 4, 0, 4, 0, 4, \ldots$ – navštívíme pouze **2 z 8** pozic!

Krok $h_2(k) = 3$ (nesoudělné s 8):

Pozice: $0, 3, 6, 1, 4, 7, 2, 5$ – navštívíme **všech 8** pozic

Shrnutí metod probingu

1. Lineární probing

- Pozice: $i, i+1, i+2, i+3, \ldots$

2. Kvadratický probing

- Pozice: $i, i+1^2, i+2^2, i+3^2, \ldots$

3. Dvojité hashování

- Pozice: $i, i+f(k), i+2 \cdot f(k), \ldots$
  kde $f(k)$ je jiná hashovací funkce

Operace s hashovací tabulkou

Vložení prvku

1. Vypočteme index $i = h(k) \bmod m$
2. Je pozice i volná?
   - **Ano** $\rightarrow$ uložíme prvek
   - **Ne** $\rightarrow$ kolize – použijeme řetězení nebo probing

Vyhledání prvku

1. Vypočteme index $i = h(k) \bmod m$
2. Je na pozici i hledaný klíč?
   - **Ano** $\rightarrow$ vrátíme hodnotu
   - **Ne** $\rightarrow$ pokračujeme podle metody řešení kolizí
3. Pokud narazíme na **prázdné místo** $\rightarrow$ prvek v tabulce není

Smazání prvku

Jak smazat prvek z hashovací tabulky?

Řetězení: jednoduše odebereme prvek ze spojového seznamu

Otevřené adresování: tady nastává problém…

Smazání prvku

Mějme prvky na pozicích i_1, i_2, i_3 (vznikly kolizemi)

Pokud odstraníme prvek na pozici i_2:

- Při vyhledávání najdeme prázdné místo na i_2
- Myslíme si, že prvek neexistuje
- Ale prvek může být na i_3!

Náhrobek (tombstone)

Místo smazání označíme pozici jako **smazanou** – použijeme **náhrobek** (tombstone)

Při **vyhledávání**: náhrobek ignorujeme a pokračujeme na další pozici

Při **vkládání**: náhrobek můžeme přepsat novým prvkem

Load factor a rehashing

Load factor

Load factor (faktor zaplnění) udává, jak moc je tabulka zaplněná

$$\alpha = \frac{\text{počet prvků}}{\text{velikost tabulky}}$$

- Vyšší load factor = více kolizí = pomalejší operace
- Optimální hodnota: přibližně 0.7 (70% zaplnění)

Rehashing

Když load factor přesáhne limit:

- Vytvoříme **větší tabulku** (typicky 2x)
- **Přehashujeme** všechny prvky – všechny se musí přepočítat a přesunout

Připomíná vám to něco? **Dynamické pole** – stejný princip zdvojnásobení

Amortizovaná složitost vkládání zůstává $\mathcal{O}^*(1)$

Složitost

Složitost operací

Operace	Průměrný případ	Nejhorší případ
Vyhledání	$\mathcal{O}(1)$	$\mathcal{O}(n)$
Vložení	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$
Smazání	$\mathcal{O}(1)$	$\mathcal{O}(n)$

Složitost operací

Průměrně $\mathcal{O}(1)$ / $\mathcal{O}^*(1)$ – proč?

Hashovací funkce nás pošle přímo na správný index

Nejhorší případ $\mathcal{O}(n)$ – proč?

Všechny klíče mají stejný hash (všechny v jednom bucketu/clusteru)

Srovnání s ostatními strukturami

Operace	Pole (neseřazené)	Pole (seřazené)	Hash tabulka
Vyhledání	$\mathcal{O}(n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Vložení	$\mathcal{O}^*(1)$	$\mathcal{O}(n)$	$\mathcal{O}^*(1)$
Smazání	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$

Srovnání s ostatními strukturami

Hashovací tabulka je **nejrychlejší** pro všechny základní operace (v průměrném případě)

Nevýhody: vyšší paměťová náročnost, neuspořádaná data, kvalita závisí na hashovací funkci

Kde se hashovací tabulky používají?

- **Slovníky** v programovacích jazycích – Python `dict`, Java `HashMap`, C# `Dictionary`
- **Cache** – rychlé ukládání často používaných dat
- **Tabulka symbolů** v překladači – mapování názvů proměnných na adresy v paměti
- **Detekce duplicit** – kontrola, zda jsme prvek již viděli
- **Počítání frekvencí** – histogram, statistiky
- **Implementace množin** (Set) – množiny bez duplicit

Kryptografické hashovací funkce

Jiný typ hashování

Hashovací funkce pro tabulky a kryptografické hashovací funkce mají **odlišné požadavky**

Pro tabulky: rychlá, uniformní distribuce

Kryptografické – navíc:

- **Jednosměrnost** – z hashe nelze zpětně odvodit vstup
- **Odolnost proti kolizím** – prakticky nemožné najít dva vstupy se stejným hashem
- **Lavinový efekt** – malá změna vstupu = velká změna hashe

Příklady kryptografických hashů

Hashe pro řetězec “Hello World”:

- MD5 = `b10a8db164e0754105b7a99be72e3fe5`
- SHA-1 = `0a4d55a8d778e5022fab701977c5d840bbc486d0`
- SHA-256 = `a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e`

Příklady kryptografických hashů

MD5 generuje **128-bitový** hash, SHA-1 **160-bitový** a SHA-256 **256-bitový**

Proč tedy délka hashe MD5 je 32 znaků, což odpovídá $32 \cdot 8 = 256$ bitům?

Výsledky jsou v **šestnáctkové soustavě** – každý znak reprezentuje 4 bity

$32 \cdot 4 = 128$ bitů

Narozeninový paradox

Narozeninový paradox

Nehledáme někoho s **konkrétním** datem narození – hledáme **jakoukoliv** shodu ve skupině

Kolik lidí musí být v místnosti, aby pravděpodobnost, že **nějací 2** mají stejné narozeniny, byla větší než 50%?

10? 23? 50? 100? 183? 365?

Pouze **23 lidí**

Narozeninový paradox – výpočet

Předpokládejme, že všichni vybraní lidé mají narozeniny v **různých** dnech

- 1. člověk má 365 možností
- 2. člověk má 364 možností
- 3. člověk má 363 možností
- ...

$$P(\text{bez kolize}) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdot \ldots \cdot \frac{365-(n-1)}{365}$$

Narozeninový paradox – výpočet

Doplněk tohoto náhodného jevu je pravděpodobnost kolize – tedy pravděpodobnost, že minimálně 2 lidé mají stejné narozeniny

$P(\text{kolize}) = 1 - P(\text{bez kolize})$

Dosadíme pro $n = 23$:

Narozeninový paradox – výpočet

$P(\text{bez kolize}) = \frac{365}{365} \cdot \frac{364}{365} \cdot \frac{363}{365} \cdot \ldots \cdot \frac{365-22}{365}$

Čitatel je klesající součin: $365 \cdot 364 \cdot 363 \cdot \ldots \cdot 343$

To lze zapsat jako: $\frac{365!}{(365-23)!} = \frac{365!}{342!}$

Jmenovatel: $\underbrace{365 \cdot 365 \cdot \ldots \cdot 365}_{23 \text{ krát}} = 365^{23}$

$P(\text{bez kolize}) = \frac{365!}{342! \cdot 365^{23}}$

Narozeninový paradox – výpočet

$P(\text{bez kolize}) = \frac{365!}{342! \cdot 365^{23}} \approx 0.4927$

$P(\text{kolize}) = 1 - 0.4927 \approx 0.5073 > 50\%$

Stačí tedy pouhých **23 lidí**, aby pravděpodobnost shody narozenin překročila 50%

Proč to pomáhá útočníkům?

Klíčový rozdíl – hledáme **konkrétní** kolizi, nebo **libovolnou** kolizi?

Konkrétní kolize: najít vstup y takový, že $h(y) = h(x)$ pro dané $x \to 2^n$ pokusů

Libovolná kolize: najít x, y takové, že $h(x) = h(y) \to 2^{n/2}$ pokusů

Proč to pomáhá útočníkům?

Proč stačí $2^{n/2}$ pokusů?

Představme si jednoduchý příklad: hash funkce s **10** možnými hodnotami (0–9)

Vygenerujeme **4 hashe**: 3, 7, 1, 7 $\rightarrow$ kolize!

Neporovnáváme každý hash s jedním konkrétním – porovnáváme **každý s každým**

Dvojice k porovnání: (3,7), (3,1), (3,7), (7,1), (7,7), (1,7) – celkem **6 dvojic**

Proč to pomáhá útočníkům?

Obecně: z k hashů vytvoříme $\binom{k}{2} = \frac{k \cdot (k-1)}{2}$ dvojic

Ze 4 hashů: $\binom{4}{2} = \frac{4 \cdot 3}{2} = 6$ dvojic

Počet dvojic roste **kvadraticky** – 2× více hashů = 4× více dvojic

Proč to pomáhá útočníkům?

Ověřme si to na narozeninách:

365 možných hodnot (dnů v roce) $\rightarrow \sqrt{365} \approx 19$

Skutečná hodnota pro 50% pravděpodobnost: **23**

$\sqrt{365} \approx 19$ je dobrý odhad – správný řád, jen mírně podhodnocuje

Platí obecné pravidlo: pro kolizi s pravděpodobností $> 50\%$ stačí přibližně $\sqrt{n}$ pokusů

Proč to pomáhá útočníkům?

Závěr pro kryptografii:

Útočník nemusí zkoušet všech 2^n možností

Stačí mu přibližně $\sqrt{2^n} = 2^{n/2}$ – tedy **odmocnina** z původního počtu

Hash funkce	Délka hashe	Pokusů pro kolizi
MD5	2^{128}	$\approx 2^{64}$
SHA-1	2^{160}	$\approx 2^{80}$
SHA-256	2^{256}	$\approx 2^{128}$

Prolomené hashovací funkce

MD5 bylo prolomeno v roce 2004

Aktuálně je považováno za zastaralé a nebezpečné

Hledání kolize v MD5 dnes trvá pouze několik sekund

SHA-1 bylo prolomeno v roce 2017

Hledání kolize trvá několik dní na superpočítači

Aktuálně bezpečné: SHA-256, SHA-3, BLAKE3

Využití kryptografických hashů

- **Ukládání hesel** – databáze neuchovává heslo, ale jeho hash
- **Kontrolní součty** – ověření integrity dat (stahování souborů)
- **Digitální podpisy** – ověření autora dokumentu
- **Git** – identifikace commitů pomocí SHA-1